

HDOC DAY-8 ACTIVITY REPORT

C Programming – Practical Report

Submitted by

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16
Date	01 September 2026

Question 1: Height of a Cylinder Formed by Melting a Sphere

Step 1 – Problem Understanding

A solid metal sphere of a given radius is melted down and recast into a solid cylinder of a given radius. Since melting and recasting does not change the amount of metal, the volume of the sphere equals the volume of the cylinder. Using this fact, the program calculates the height of the resulting cylinder.

Required Input(s):

- Radius of the sphere, r (float)
- Radius of the cylinder, R (float)

Required Output:

- Height of the cylinder, h (float)

Formula:

- Volume of sphere = $(4/3) \times \pi \times r^3$
- Volume of cylinder = $\pi \times R^2 \times h$
- Equating volumes: $(4/3) \times \pi \times r^3 = \pi \times R^2 \times h$
- $\Rightarrow h = (4 \times r^3) / (3 \times R^2)$

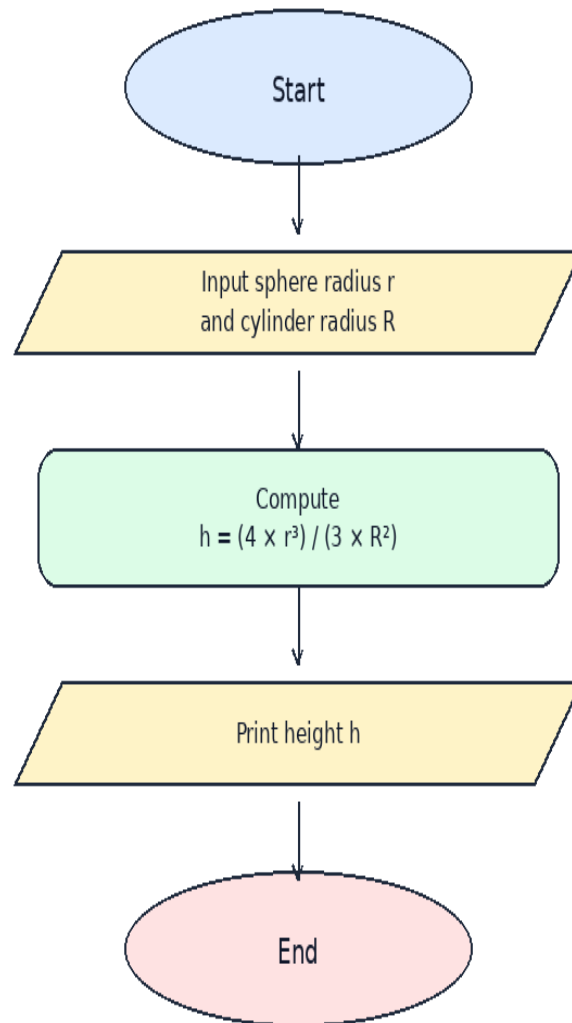
Step 2 – Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	r = 3, R = 2	h = 9.00	9.00	Pass
2	r = 6, R = 3	h = 32.00	32.00	Pass
3	r = 4.5, R = 3	h = 13.50	13.50	Pass

Step 3 – Algorithm

1. Start.
2. Read the sphere radius r from the user.
3. Read the cylinder radius R from the user.
4. Compute $h = (4 \times r \times r \times r) / (3 \times R \times R)$.
5. Display the computed height h.
6. Stop.

Evaluation against test cases: Tracing the algorithm by hand for each test case in Step 2 with the given inputs reproduces the same Expected Output values shown above, confirming the algorithm's logical correctness.

Step 4 – Flowchart**Flowchart - Q1: Height of Cylinder**

Evaluation against test cases: Following the flowchart path (Start → Input → Compute → Print → End) for each test case yields the same results as the algorithm and the Expected Output column, so the flowchart is consistent with Steps 2 and 3.

Step 5 – C Program

```
#include <stdio.h>

int main() {
    float r, R, h;

    printf("Enter the radius of the sphere (r): ");
    scanf("%f", &r);

    printf("Enter the radius of the cylinder (R): ");
    scanf("%f", &R);

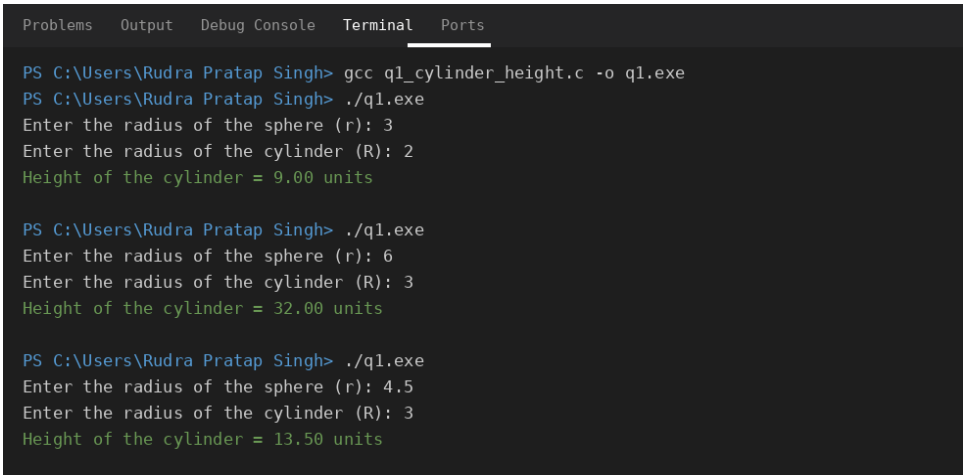
    /* Volume of sphere = Volume of cylinder (melting conserves volume)
       (4/3) * PI * r^3 = PI * R^2 * h
       => h = (4 * r^3) / (3 * R^2) */
    h = (4 * r * r * r) / (3 * R * R);

    printf("Height of the cylinder = %.2f units\n", h);

    return 0;
}
```

Step 6 – Compilation, Execution & Evaluation

The program was compiled with GCC (gcc q1_cylinder_height.c -o q1.exe) with no errors or warnings, and executed three times using the inputs from Step 2.



```
PS C:\Users\Rudra Pratap Singh> gcc q1_cylinder_height.c -o q1.exe
PS C:\Users\Rudra Pratap Singh> ./q1.exe
Enter the radius of the sphere (r): 3
Enter the radius of the cylinder (R): 2
Height of the cylinder = 9.00 units

PS C:\Users\Rudra Pratap Singh> ./q1.exe
Enter the radius of the sphere (r): 6
Enter the radius of the cylinder (R): 3
Height of the cylinder = 32.00 units

PS C:\Users\Rudra Pratap Singh> ./q1.exe
Enter the radius of the sphere (r): 4.5
Enter the radius of the cylinder (R): 3
Height of the cylinder = 13.50 units
```

The Actual Output for all three test cases exactly matches the Expected Output, so all test cases Pass and the program is verified as correct.

Question 2: Total Boundary Length of a Rectangle with Two Semicircular Ends

Step 1 – Problem Understanding

A shape (similar to an athletics track or a 'stadium' shape) is formed from a rectangle whose two shorter sides are replaced by semicircles. The two semicircles, each of radius equal to half the rectangle's width, together form one full circle. The program calculates the total boundary (perimeter) length of this shape.

Required Input(s):

- Length of the rectangle, length (float)
- Width of the rectangle, width (float) — also the diameter of each semicircular end

Required Output:

- Total boundary length, perimeter (float)

Formula:

- Two straight sides contribute: $2 \times \text{length}$
- Two semicircular ends (radius = width / 2) together form one full circle of circumference $\pi \times \text{width}$
- $\Rightarrow \text{perimeter} = (2 \times \text{length}) + (\pi \times \text{width})$

Step 2 – Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	length = 10, width = 4	32.57	32.57	Pass
2	length = 15, width = 6	48.85	48.85	Pass
3	length = 8, width = 8	41.13	41.13	Pass

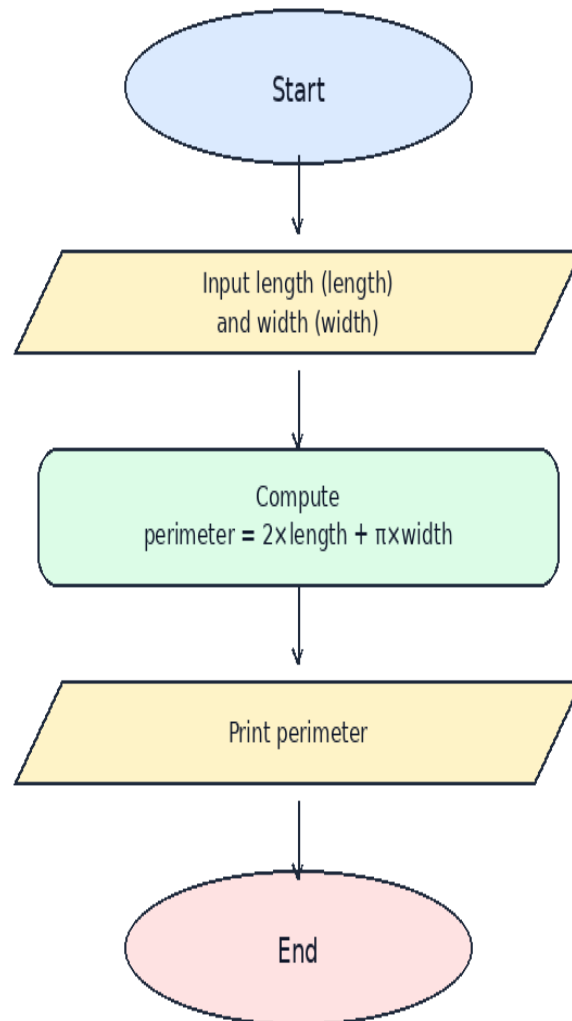
Step 3 – Algorithm

1. Start.
2. Read the rectangle length (length) from the user.
3. Read the rectangle width (width) from the user.
4. Compute perimeter = $(2 \times \text{length}) + (\pi \times \text{width})$, using $\pi \approx 3.14159$.
5. Display the computed perimeter.
6. Stop.

Evaluation against test cases: Tracing the algorithm by hand for each test case in Step 2 with the given inputs reproduces the same Expected Output values shown above, confirming the algorithm's logical correctness.

Step 4 – Flowchart

Flowchart - Q2: Boundary Length



Evaluation against test cases: Following the flowchart path (Start → Input → Compute → Print → End) for each test case yields the same results as the algorithm and the Expected Output column, so the flowchart is consistent with Steps 2 and 3.

Step 5 – C Program

```
#include <stdio.h>

#define PI 3.14159

int main() {
    float length, width, perimeter;

    printf("Enter the length of the rectangle: ");
    scanf("%f", &length);

    printf("Enter the width of the rectangle: ");
    scanf("%f", &width);

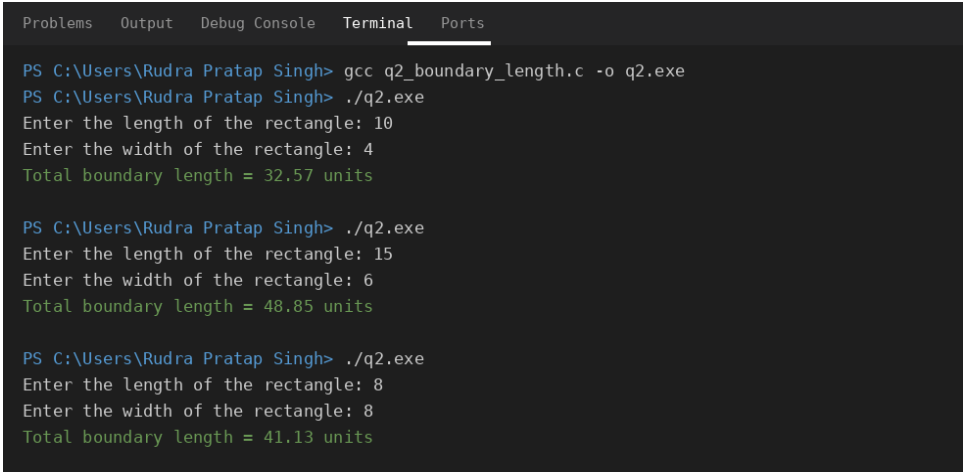
    /* The two semicircular ends (each of radius = width/2) together
       form one full circle of diameter = width.
       Boundary length = 2 * length (two straight sides) + PI * width */
    perimeter = (2 * length) + (PI * width);

    printf("Total boundary length = %.2f units\n", perimeter);

    return 0;
}
```

Step 6 – Compilation, Execution & Evaluation

The program was compiled with GCC (gcc q2_boundary_length.c -o q2.exe) with no errors or warnings, and executed three times using the inputs from Step 2.



```
PS C:\Users\Rudra Pratap Singh> gcc q2_boundary_length.c -o q2.exe
PS C:\Users\Rudra Pratap Singh> ./q2.exe
Enter the length of the rectangle: 10
Enter the width of the rectangle: 4
Total boundary length = 32.57 units

PS C:\Users\Rudra Pratap Singh> ./q2.exe
Enter the length of the rectangle: 15
Enter the width of the rectangle: 6
Total boundary length = 48.85 units

PS C:\Users\Rudra Pratap Singh> ./q2.exe
Enter the length of the rectangle: 8
Enter the width of the rectangle: 8
Total boundary length = 41.13 units
```

The Actual Output for all three test cases exactly matches the Expected Output, so all test cases Pass and the program is verified as correct.

(Note – Thoery is by AI. To be exact)